

Apuntadores

Estructura de Datos y Algoritmos I

Facultad de Ingeniería
Elba Karen Sáenz García

EDAI- Elba Karen Sáenz García- P UNAM

Apuntadores

- Un **apuntador o puntero** es una variable que puede almacenar direcciones de memoria (variables o datos).

Declaración en lenguaje C

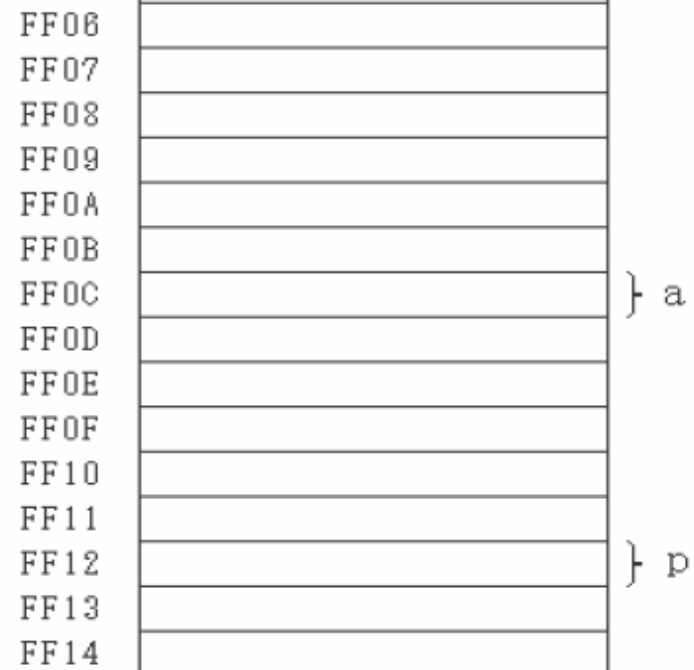
```
char *pa;
```

```
int *pi;
```

```
float *pf;
```

■ Ejemplo uso

```
char a, *p;
```



```
a = 'c';  
p = &a;
```

FF06
FF07
FF08
FF09
FF0A
FF0B
FF0C
FF0D
FF0E
FF0F
FF10
FF11
FF12
FF13
FF14

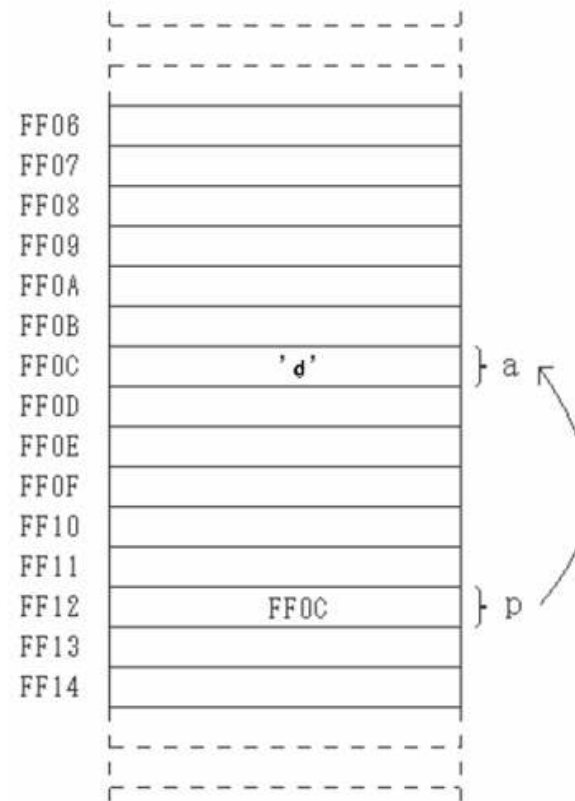
'c'

FF0C

} a

} p

`*p='d';`



Operadores Importantes

- **Dirección (&):** Obtiene la dirección de memoria de la variable de la variable a la que precede.
- **Indirección: (*)** Si se antepone a una variable apuntador se obtiene el valor que contiene la dirección de memoria a la que hace referencia el apuntador(a lo que apunta).

$a + n = \text{Dirección almacenada} + n * \text{tamaño_tipo_dato}$

Aritmética de
apuntadores
Sumar o restar
un número
entero a un
puntero.

```
short int n, *ap, *bp;
n=1;
ap = &n;
```

ff00	1	n
ff01		
ff02		
ff03		
ff04		
ff05		
ff06		
ff07		
ff08		bp
ff09	ff00	ap
ff0a		

Aritmética de apuntadores

Ff00+2(2)

bp = ap + 2

ff00	1	n
ff01		
ff02		
ff03		
ff04		
ff05		
ff06		
ff07		
ff08	ff04	bp
ff09	ff00	ap
ff0a		

ap+2

Otro ejemplo

```
char n,*ap,*bp;
```

```
n=3;
```

```
ap=&n;
```

```
bp=ap+2;
```

```
ap=ap+3
```

```
*ap=3;
```

ff00	1	n
ff01		
ff02		
ff03	3	
ff04		
ff05		
ff06		
ff07		
ff08	ff02	bp
ff09	ff03	ap
ff0a		

← ap

```
#include <stdio.h>

main() {
    char valor;
    char *p;
    valor = 97;
    printf("\n %p => | %d | <= dirección y datos de valor. \n", &valor, valor);  FF00, 97
    p = &valor;  (p tiene FF00)

    printf(" %p => | %p | <= dirección y datos del apuntador. \n", &p, p);  FF12 , FF00
    printf("\n Valor almacenado en apuntador = %p \n", p);
    printf(" Dirección de apuntador : &p = %p\n", &p);
    printf(" Valor referenciado por apuntador: *p = %d\n", *p);
}
```

Observaciones Apuntadores

***Un apuntador puede asignarse a
otro apuntador***

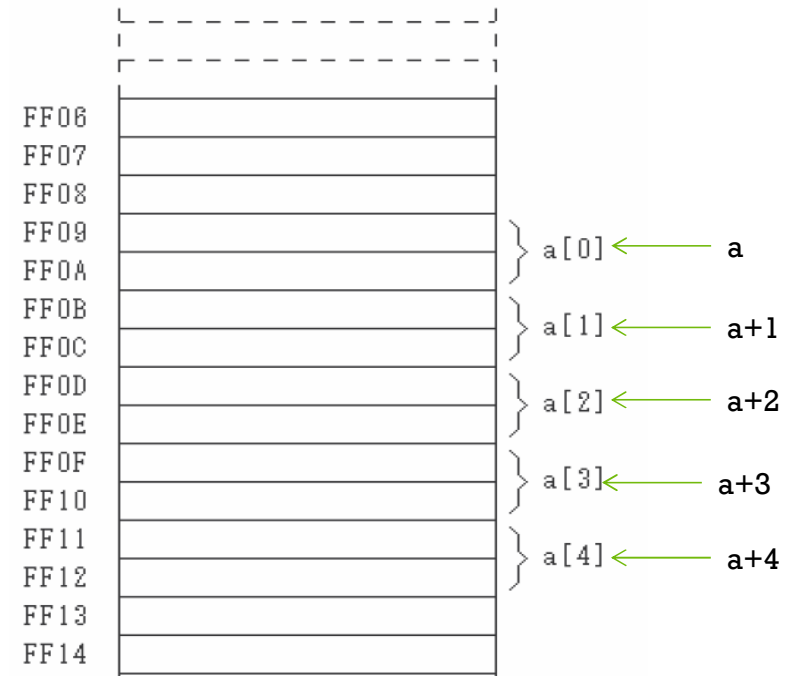
```
long a,*p,*q;   p=&a;   q=p;
```

```
#include<stdio.h>

int main(){
int var,*p,i;
var=0;
p=&var;
printf("El valor inicial de var es: %d\n",var);
for(i=0;i<6;i++){
    printf("nuevo valor: ");
    scanf("%d",p);
    printf("var ha cambiado por: %d\n",var);
}
}
```

Relación entre un arreglo y un apuntador

- **Relación entre los apuntadores y los arreglo unidimensionales**
 - Se pueden direccionar arreglos como si fueran apuntadores y apuntadores como si fueran arreglos.
- En el lenguaje de programación C, el nombre de un arreglo es el índice a la dirección de inicio del primer elemento del arreglo.



a + 0	apunta a	a[0]	que corresponde a la dirección FF09
a + 1	apunta a	a[1]	que corresponde a la dirección FF0B
a + 2	apunta a	a[2]	que corresponde a la dirección FF0D
a + 3	apunta a	a[3]	que corresponde a la dirección FF0F
a + 4	apunta a	a[4]	que corresponde a la dirección FF11

- Si se quieren modificar valores

Forma 1

```
*(a+0)=5; *(a+1)=6; *(a+2)=10; *(a+3)=11; *(a+4)=6;
```

Forma 2:

```
for(i=0; i<5; i++)
```

```
    scanf("%d", (a+i));
```

- **Imprimir contenido arreglo mediante el apuntador**

```
for(i=0; i<5; i++)
```

```
    printf("%d", *(a+i));
```

Ejemplo

```
#include<stdio.h>

main() {
    int x[5]={1,2,3,4,5};
    int *y, renglon;

    y = x;
    printf("%d %d %d \n",x[1],*y,*y+1));
    for(renglon =0; renglon <= 4; renglon ++)
        printf("%d\t",*(y+renglon));
}
```


The slide features a light gray background with several thin, curved lines in both solid and dashed styles, primarily concentrated on the left and bottom right sides. A large green speech bubble is positioned on the left, containing the word 'Funciones'. To its right is a bulleted list of three items: '■ Prototipos', '■ Definición', and '■ Llamada'. The bottom right corner contains the text 'EDAI- Elba Karen Sáenz García- F UNAM'.

Funciones

- **Prototipos**
- **Definición**
- **Llamada**

Funciones en C definición

```
tipo_de_retorno nombre_de_la_función (lista_de_parámetros)
{
    cuerpo_de_la_función
    return expresión
}
```

```
#include <stdio.h>
```

```
float precio(float base, float impuesto); /* declaración */
```

```
main()
{
    float importe = 2.5;
    float tasa = 0.07;

    printf("El precio a pagar es: %.2f\n", precio(importe, tasa));
    return 0;
}
```

```
float precio(float base, float impuesto) /* definición */
{
    float calculo;
    calculo = base + (base * impuesto);
    return calculo;
}
```

MANEJO DE Paso de Parámetros EN C

- **Paso de parámetros por valor:** Se hace una copia del valor de los parámetros para trabajarlos en la función. Los parámetros no se modifican.
- **Paso de parámetros por referencia:** Se utiliza la referencia o dirección para poder modificar los parámetros. O bien, en C se hace una copia de la dirección de memoria de la variable que se quiere modificar

```
//////  
#include <stdio.h>
```

```
void modificar(int variable);
```

```
main()
```

```
{
```

```
    int i = 1;
```

```
    printf("\ni=%d antes de llamar a la función modificar", i);
```

```
    modificar(i);
```

```
    printf("\ni=%d después de llamar a la función modificar", i);
```

```
}
```

```
void modificar(int variable)
```

```
{
```

```
    printf("\nvariable = %d dentro de modificar", variable);
```

```
    variable = 9;
```

```
    printf("\nvariable = %d dentro de modificar", variable);
```

```
}
```

Ejemplo

```
void swap(int a, int b);  
main(){  
    int c=2,d=5;  
    swap(c,d);  
}  
void swap(int a, int b){  
    int tmp;  
    tmp=a;  
    a=b;  
    b=tmp;  
}
```

Ejemplo

```
void swap(int *a, int *b);  
main(){  
    int c=2,d=5;  
    swap(&c,&d);  
}  
void swap(int *a, int *b){  
    int tmp;  
    tmp=*a;  
    *a=*b;  
    *b=tmp;  
}
```

Inicialización de apuntadores

- Los punteros deben ser inicializados.
- Utilizar el identificador NULL (definido en `stdio.h`) para indicar que el puntero no apunta a nada.
- El 0 es el único valor entero que puede asignarse directamente a un puntero y es equivalente a NULL.
- Cuando se asigna 0 a un puntero se realiza un casting previo automático al tipo apropiado.

Apuntadores void

Sintaxis

```
void * VoidPtr;
```

Un *puntero a void* es un **puntero genérico**, que puede recibir el valor de cualquier otro puntero incluso NULL

Es decir que puede apuntar a objetos de cualquier tipo (con algunas excepciones).

```
#include<stdio.h>

main () {
    int x = 1;
    float r = 1.0;
    void vptr ;
    vptr=&x;
    *(int *) vptr = 2;
    printf("\n%d", x);
    vptr = &r;
    *(float *)vptr = 1.1;
    printf("\n%f", r);
}
```

Doble
Apuntador o
punteros dobles

- Variables cuyos posibles valores son direcciones de otras variables apuntador .

```
1  #include<stdio.h>
2  main()
3  {
4      int a,*b,**c,***d;
5      a=2;
6      b=&a;
7      c=&b;
8      d=&c;
9      **c=3;
10     printf("a: %d ",a);
11     printf("\n Dir &a %p", &a);
12     printf("\n b  %p", b);
13     printf("\n Dir &b %p", &b);
14     printf("\n contenido apuntaba %d", *b);
15     printf("\n c %p", c);
16     printf("\n contenido *c a %p", *c);
17     printf("\n **c: %d ",**c);
18     printf("\n **c: %d ",***d);
19     }
```